CS & IT ENGINEERING

Operating System

Process Synchronization

Lecture -1

By- Vishvadeep Gothi sir

Recap of Previous Lecture

Doubt

Topic — Multithreading

Topic — System Call: Fork()

Topics to be Covered

Topic Synchronization

Topic Race Condition

[NAT]

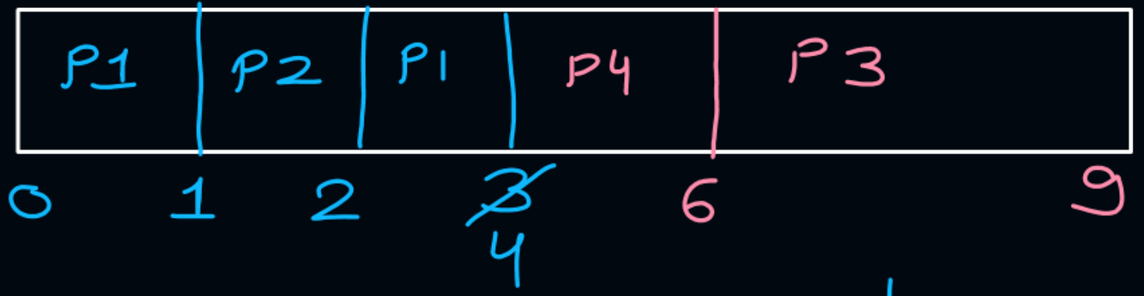
#Q. Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

Process	P1	P2	P3	P4
Arrival time	0	1	3	4
CPU burst Time	~~3~~ ~~2~~ 1	~~1~~	3	Z = 2

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is __2__.

Gate - 2019

$$\text{avg } WT = \frac{\text{Total } WT}{4} \Rightarrow \text{Total } WT = 4 \text{ ms}$$



	BT
P3	3
P4	2

	CT	TAT	WT
P1	4	4	1
P2	2	1	0
P3			1+2
P4			0
			4

[MCQ]

#Q. Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

GATE - 2006

A 0%

B 10.6%

C 30.0%

D 89.4%

[MCQ]

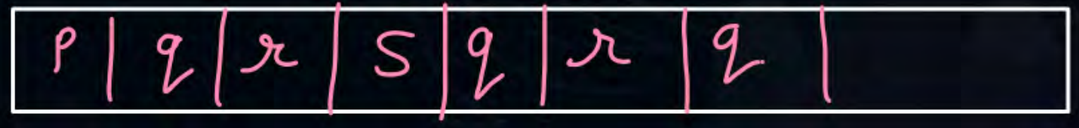
#Q. Consider four processes P, Q, R, and S scheduled on a CPU as per round-robin algorithm with a time quantum of 4 units. The processes arrive in the order P, Q, R, S, all at time t = 0. There is exactly one context switch from S to Q, exactly one context switch from R to Q, and exactly two context switches from Q to R. There is no context switch from S to P. Switching to a ready process after the termination of another process is also considered a context switch. Which one of the following is NOT possible as CPU burst time (in time units) of these processes? **GATE-2022**

(A) P = 4, Q = 10, R = 6, S = 2

(B) P = 2, Q = 9, R = 5, S = 1

(C) P = 4, Q = 12, R = 5, S = 4

(D) P = 3, Q = 7, R = 7, S = 3



Topic : Fork() System Call

Fork system call is used for creating a new process, which is called child process.

Child process runs concurrently with the process that makes the fork() call (parent process).

→ child process executes from the next block, statement or syntax of fork() which created child.

Topic : Fork() System Call

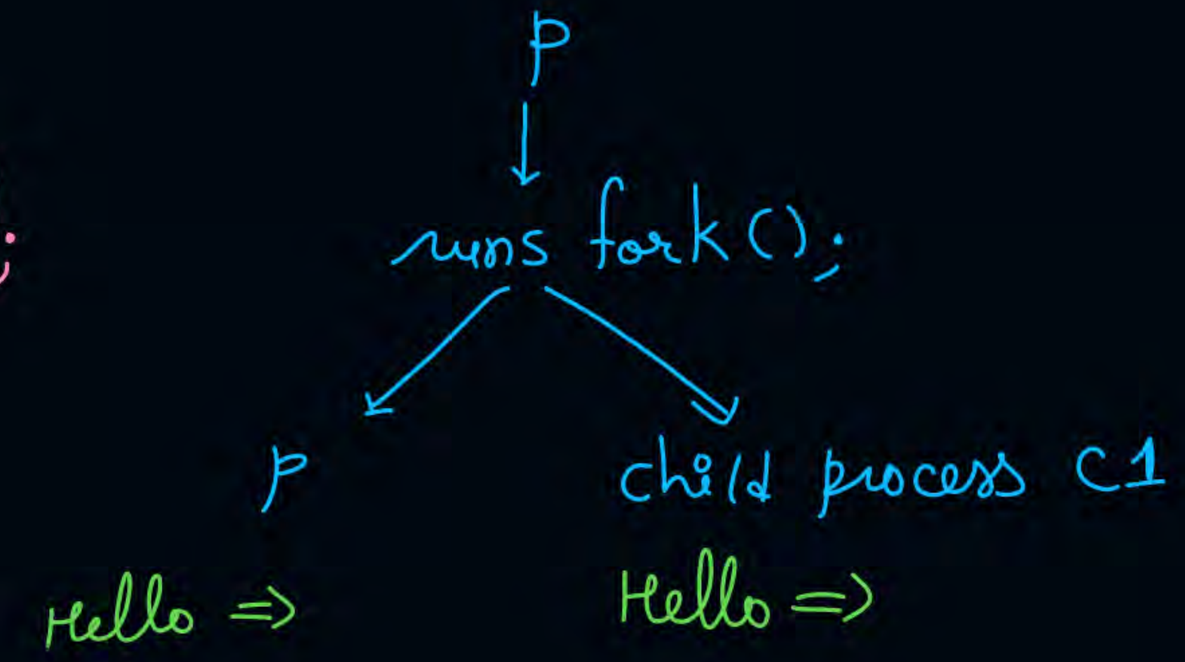
It takes no parameters and returns an integer value

- Negative Value:
  Creation of a child process was unsuccessful

- Zero:
  Returned to the newly created child process

- Positive value:
  Returned to parent or caller. The value contains process ID of newly created child process

```
fork();
printf("Hello ⇒");
```



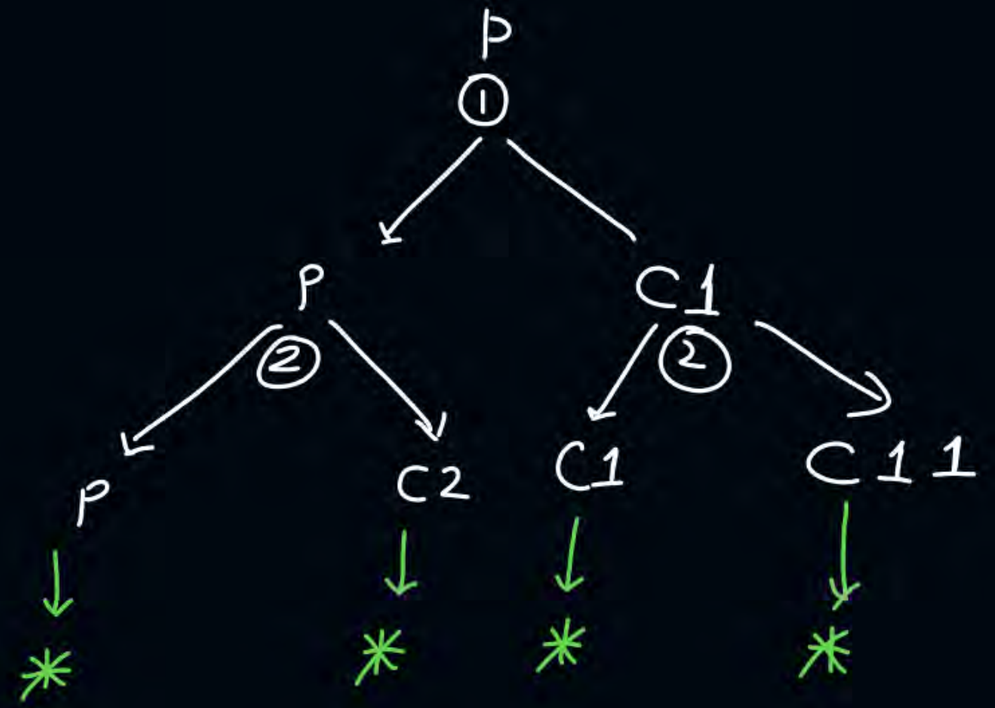
output

Hello ⇒ Hello ⇒

```
fork()1;
fork()2;
printf("*");
```

output :-

****



1 Parent
3 child processes

[MCQ]

#Q. A process executes the code

```
fork ();
fork ();
fork ();
```

Total 8 processes
1 parent, 7 child processes

The total number of child processes created is?

GATE-2008

(A) 3
(B) 4
(C) 7
(D) 8

[NAT]

#Q. A process executes the code

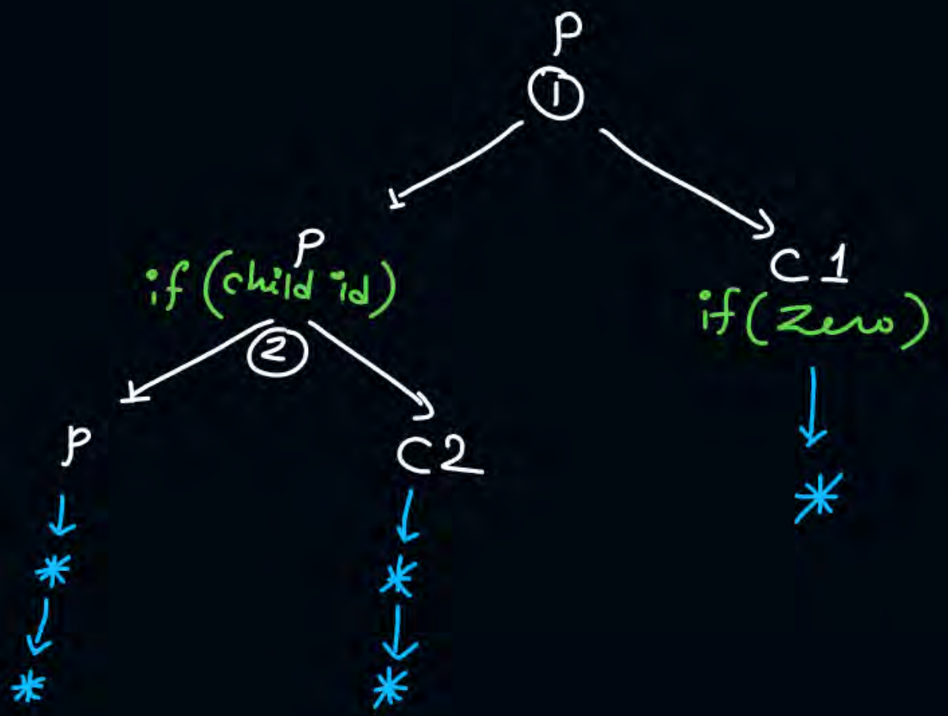
```
fork ();
:
fork ();
```

There are n such statements. The total number of child processes created is?

Total processes = 2^n

Total child processes = $2^n - 1$

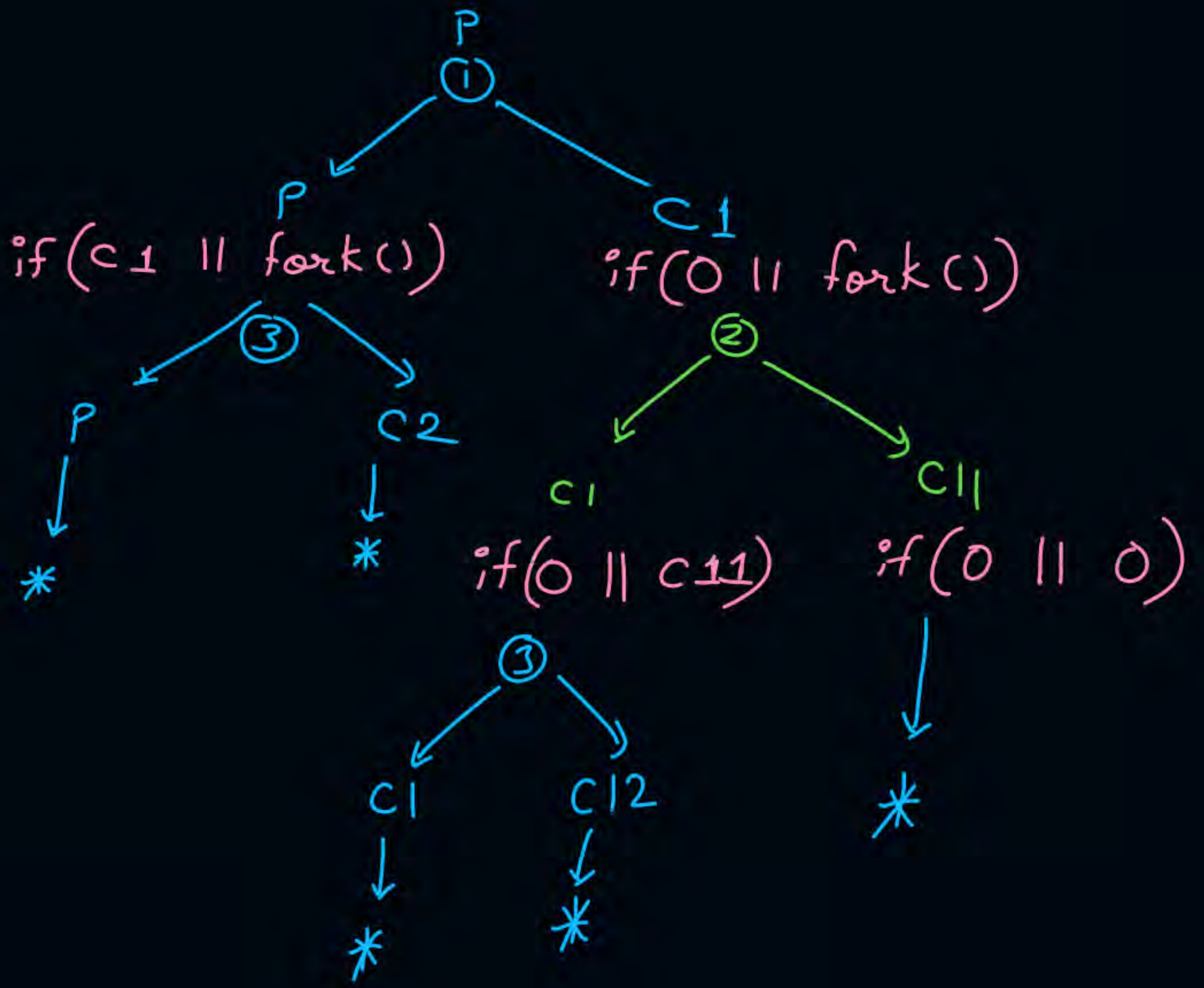
```
①
if(fork(1))
{
  ②
  fork();
  printf("*");
}
printf("*");
```



No. of times "*" is printed 5 ?

```
if ( fork() || fork() )
{
   fork();
}
printf("*");
```

no. of times * printed 5 ?



H. W.

```
if ( fork () && fork () )
{
    fork ();
}
printf (" * ");
```

no. of times * printed — ?

Topic : Wait() System Call

A call to wait() blocks the calling process until one of its child processes completes

After child process terminates, parent continues its execution after wait system call instruction

[NAT]

H.W.

#Q. Consider the following code snippet using the fork() and wait() system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```
int x = 3;
while(x > 0) {
fork();
printf("hello");
wait(NULL);
x--;
}
```

The total number of times the printf statement is executed is _____?

GATE-2024

Process synchronization

Topic : Process Types

1. Independent → Processes do not require Communication with other.

2. Cooperating/Coordinating/Communicating

⇓

Processes communicates with other processes.

IPC (Inter Process Communication) ⇒ done by OS

Topic : Need Of Synchronization

The communicating processes must execute in such a way that their final result be the desired output.

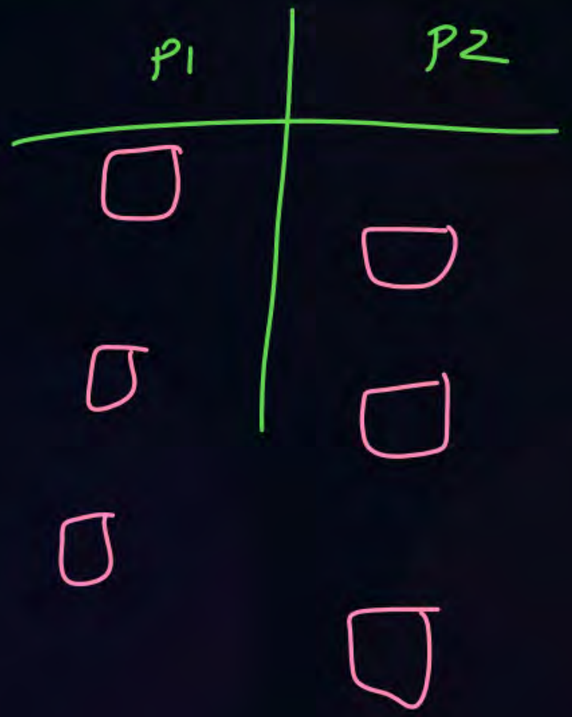
Topic : Problems Without Synchronization

Problems without Synchronization:

- Inconsistency
- Loss of Data
- Deadlock

Topic : Race Condition → Problem

A race condition is an undesirable situation, it occurs when the final result of concurrent processes depends on the sequence in which the processes complete their execution.



$x = \cancel{5}\ \cancel{7}\ 8$

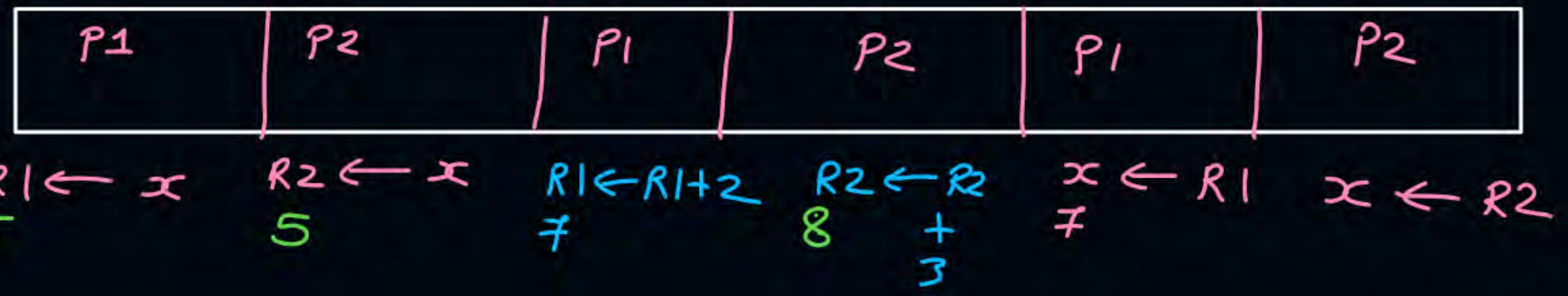
P1

$x = x + 2$

$R1 \leftarrow x$

$R1 \leftarrow R1 + 2$

$x \leftarrow R1$



P2

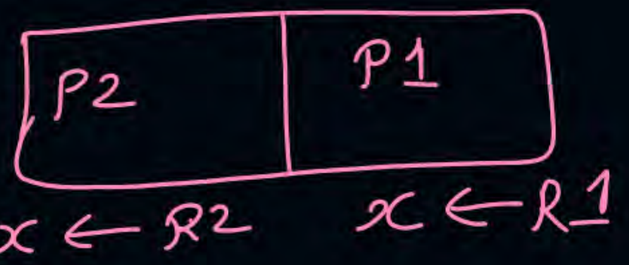
$x = x + 3$

$R2 \leftarrow x$

$R2 \leftarrow R2 + 3$

$x \leftarrow R2$

||



$x = \cancel{8}\ 7$

There are 4 possible ways to run P1 and P2

Case 1:- P1 completely runs then P2 runs ⇒ $x = 10$

Case 2:- P2 ——|| —————— ||— P1 runs ⇒ $x = 10$

Case 3:- Both P1 and P2 runs concurrently (reads x concurrently) and P1 completes last ⇒ $x = 7$

Case 4:- ——|| —————— and P2 completes last ⇒ $x = 8$

[NAT]

#Q. X = 10

P1
X = X /2
5 10

P2
X = X+4
14 10

How many different values of X are possible after both processes finish executing?

Case 1:- P1 then P2 X = 9

Case 2 :- P2 then P1 X = 7

Case 3:- concurrent read of x and P1 runs last
X = 5

Case 4:- ——|| —————— P2 runs last
X = 14

Ans = 4 (5, 7, 9, 14)

[NAT]

X = 4

#Q. The following pair of processes share a common variable X.

Process A	Process B
int Y; Y = X*2; X = Y;	int Z; Z = X+1; X = Z;

X = 9

X = 10

X = 8

X = 5

X is set to 4 before either process begins execution. As usual, statements within a process are executed sequentially, but statements in process A may execute in any order with respect to statements in process B.
How many different values of X are possible after both processes finish executing?

Ans = 4 {5, 8, 9, 10}

[NAT]

The following two functions $P1$ and $P2$ that share a variable B with an initial value of 2 execute concurrently.

```
P1() {
    C = B - 1;
    B = 2 * C;
}
```

```
P2(){
    D = 2 * B;
    B = D - 1;
}
```

$B = \{2, 3, 4\}$

The number of distinct values that B can possibly take after the execution is 3.

Ques) $x = 5$

P1	P2	P3
$x = x + 1$	$x = x + 2$	$x = x + 3$

min. possible value of x = ____ ? = P1 reads $x = 5$ and writes at the end $x = 6$

max ____ " ____ = ____ ? ⇒ Run all processes one after another

$x = 11$

distinct possible values of x = 6 ?

$\{6, 7, 8, 9, 10, 11\}$

5 + 1 + 2 + 3

6, 7, 8

9, 10, 11

[NAT]

H.W.

Consider three concurrent processes P1, P2 and P3 as shown below, which access a shared variable D that has been initialized to 100.

P1	P2	P3
: : D = D + 20 : :	: : D = D – 50 : :	: : D = D + 10 : :

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of Y – X is _____.

2 mins Summary

Topic Synchronization

Topic Race Condition

Happy Learning

THANK - YOU